

Sparse Matrix Multiplication Using CSR Format

Aizaz Rafique

The `sparse_multiply()` function efficiently performs sparse matrix-vector multiplication using the **Compressed Sparse Row (CSR)** representation. The function scans a dense row-major matrix A, extracts all non-zero elements into CSR format, and computes the product:

$$y = A \times x$$

The function prototype is:

```
1 void sparse_multiply(  
2     int rows,  
3     int cols,  
4     const double* A,  
5     const double* x,  
6     int* out_nnz,  
7     double* values,  
8     int* col_indices,  
9     int* row_ptrs,  
10    double* y  
11 );
```

The parameters `rows` and `cols` specify the dimensions of matrix A. The matrix is stored in row-major order and accessed through the pointer A, while `x` is the input vector used in multiplication. The arrays `values`, `col_indices`, and `row_ptrs` are caller-provided buffers used to store the CSR representation of the matrix. The variable `out_nnz` stores the total number of non-zero elements, and `y` stores the final output vector.

The function begins by initializing the non-zero counter:

```
1 int nnz = 0;
```

The matrix is then scanned row by row using nested loops:

```
1 for(int i = 0; i < rows; i++) {  
2     row_ptrs[i] = nnz;  
3  
4     for(int j = 0; j < cols; j++) {  
5         double scanA = A[i * cols + j];  
6  
7         if(scanA != 0.0) {  
8             values[nnz] = A[i * cols + j];  
9             col_indices[nnz] = j;  
10            nnz++;  
11        }  
12    }
```

13

```
}
```

The expression:

```
1 A[i * cols + j]
```

is used to access matrix elements in row-major format. Whenever a non-zero element is found, its value is stored in the `values[]` array and its column index is stored in `col_indices[]`. The variable `nnz` keeps track of the total number of non-zero entries encountered so far.

Before processing each row, the current value of `nnz` is stored in:

```
1 row_ptrs[i] = nnz;
```

This marks the starting position of that row inside the CSR arrays. After the entire matrix is scanned, the final row boundary is stored as:

```
1 row_ptrs[rows] = nnz;  
2 *out_nnz = nnz;
```

The CSR representation therefore consists of:

- `values[]` stores all non-zero values
- `col_indices[]` stores column positions of non-zero values
- `row_ptrs[]` stores row starting indices

After building the CSR structure, the function performs sparse matrix-vector multiplication. For every row, only the non-zero elements are processed:

```
1 for(int i = 0; i < rows; i++) {  
2     double sum = 0;  
3  
4     for(int k = row_ptrs[i]; k < row_ptrs[i + 1]; k++) {  
5         sum += values[k] * x[col_indices[k]];  
6     }  
7  
8     y[i] = sum;  
9 }
```

The loop:

```
1 for(int k = row_ptrs[i]; k < row_ptrs[i + 1]; k++)
```

iterates only through the non-zero elements belonging to row `i`. Each non-zero value is multiplied with the corresponding vector element:

```
1 values[k] * x[col_indices[k]]
```

and accumulated into `sum`. The final result for each row is stored in:

```
1 y[i] = sum;
```

The conversion from dense format to CSR requires scanning all matrix elements, giving a time complexity of:

$$O(\text{rows} \times \text{cols})$$

The multiplication phase processes only non-zero elements, resulting in:

$$O(nnz)$$

where **nnz** is the number of non-zero entries in the matrix. By avoiding operations on zero values, the CSR-based approach significantly improves efficiency for sparse matrices and reduces unnecessary computations.